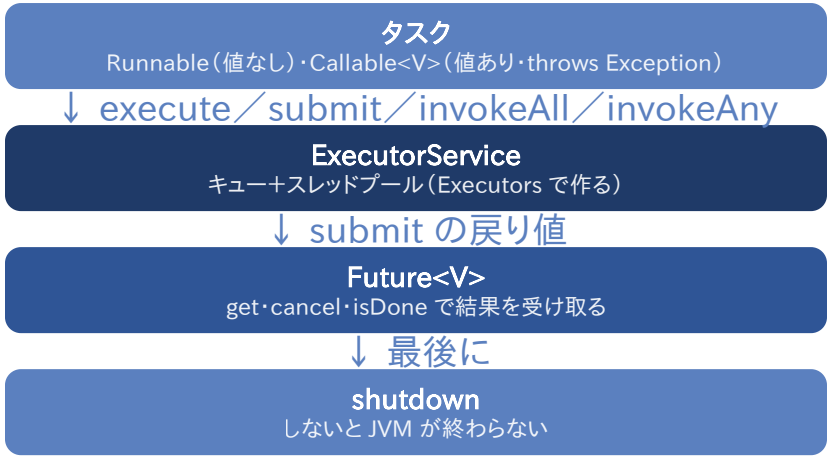
Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 5 並列処理　#13

ExecutorServiceと並行コレクション

ExecutorService・Future・Callable、CopyOnWriteArrayList・ConcurrentHashMap・BlockingQueue、CountDownLatch・CyclicBarrier・Flow

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java SE 17 API の java.util.concurrent パッケージ（ExecutorService・Executors・Future・CopyOnWriteArrayList・ConcurrentHashMap・BlockingQueue・CountDownLatch・CyclicBarrier・Flow・SubmissionPublisher）、The Java Tutorials の Executors・Concurrent Collections）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 スレッドを自分で作らない



部品	代表
実行のしくみ	ExecutorService・ScheduledExecutorService
並行コレクション	CopyOnWriteArrayList・ConcurrentHashMap・ConcurrentSkipListMap・BlockingQueue
同期の道具	CountDownLatch・CyclicBarrier
フロー制御	Flow（Publisher・Subscriber・Subscription）・SubmissionPublisher

すべて java.util.concurrent パッケージ。

出典：試験内容チェックリスト 5／API java.util.concurrent パッケージの説明

第1章

ExecutorService と Future

タスクを出して、結果を受け取る。

Executors で作る

メソッド	スレッド	使いどころ
newSingleThreadExecutor()	1 つ	出した順に 1 つずつ実行
newFixedThreadPool(n)	n 個で固定	同時に n 個まで、残りはキューで待つ
newCachedThreadPool()	必要なだけ作る	空いたスレッドを再利用（60 秒使わないと消える）
newScheduledThreadPool(n)	n 個	遅らせて・くり返し実行

スレッド名は「pool-プール番号-thread-番号」（例：pool-2-thread-1）。

プールのスレッドはデーモンでない。shutdown しないと main が終わっても JVM が終わらない。

出典：API Executors・Executors.defaultThreadFactory／JDK 17 での実行結果

execute と submit

```
ExecutorService es = Executors.newSingleThreadExecutor();
es.execute(() -> print("A"));          // void
Future<?> f = es.submit(() -> print("B"));
Future<Integer> g = es.submit(() -> 42);
Future<String> h = es.submit(() -> {}, "R");
f.get()  // null
g.get()  // 42
h.get()  // R
// 表示：AB

Callable<Integer> c = () -> 1;
es.execute(c);
// × Callable<Integer> cannot be converted to Runnable
```

メソッド	戻り値
execute(Runnable)	void
submit(Runnable)	Future<?>（get は null）
submit(Runnable, T)	Future<T>（get は T）
submit(Callable<T>)	Future<T>
invokeAll(タスクの集まり)	List<Future<T>>（全部終わってから・渡した順）
invokeAny(タスクの集まり)	T（どれか 1 つの結果）

出典：API Executor.execute・ExecutorService.submit・invokeAll・invokeAny／JDK 17 でのコンパイルと実行の結果

Future で結果を受け取る

```
Future<Integer> f = es.submit(() -> {
  throw new IllegalStateException("bad"); });
try { f.get(); }
catch (ExecutionException e) {
  e.getCause()  // IllegalStateException: bad
}

Future<String> s = es.submit(() -> {
  Thread.sleep(2000); return "done"; });
s.get(100, TimeUnit.MILLISECONDS); // TimeoutException
s.cancel(true);  // true（実行中なら割り込み）
s.isCancelled(); // true
s.isDone();      // true
s.get();         // CancellationException
```

例外	いつ
ExecutionException	タスクの中で例外（getCause で元の例外）
TimeoutException	get(時間, 単位) が時間切れ
CancellationException	中止したタスクの get
InterruptedException	get で待っている間に割り込み

> isDone は、正常・例外・中止のどれで終わっても true。

出典：API Future.get・cancel・isDone、ExecutionException・CancellationException・TimeoutException／JDK 17 での実行結果

終わらせ方と時間指定

メソッド	中身
shutdown()	新しいタスクを断る。受け付け済みは最後まで実行。待たない
shutdownNow()	実行中に割り込み、始まっていないタスクを List<Runnable> で返す
awaitTermination(t, 単位)	終わるまで待つ（終われば true、時間切れは false）
isShutdown()	shutdown を呼んだら true
isTerminated()	shutdown のあと全タスクが終わったら true

> shutdown のあとの execute・submit は RejectedExecutionException。

```
ScheduledExecutorService s =
  Executors.newScheduledThreadPool(1);
s.schedule(() -> "later", 50, MILLISECONDS)
 .get();                 // later（1 回だけ）
s.scheduleAtFixedRate(r, 0, 10, MILLISECONDS);
// 開始から開始までを一定に
s.scheduleWithFixedDelay(r, 0, 10, MILLISECONDS);
// 終わりから次の開始までを一定に
// くり返しの 2 つは Runnable だけ
```

出典：API ExecutorService.shutdown・shutdownNow・awaitTermination、ScheduledExecutorService／JDK 17 での実行結果

第2章

並行コレクション

走査中に変更しても壊れない。

CopyOnWriteArrayList

```
List<String> l = new ArrayList<>(List.of("a","b","c"));
for (String s : l) if (s.equals("a")) l.add("d");
// ConcurrentModificationException

List<String> c = new CopyOnWriteArrayList<>(List.of("a","b","c"));
for (String s : c) if (s.equals("a")) c.add("d");
c   // [a, b, c, d]（d は走査に出ない）

Iterator<Integer> it = list123.iterator();
list123.add(4);
// it で数えると 3、size は 4
list123.iterator().remove();
// UnsupportedOperationException
```

書き込みのたびに配列をコピー
add・set・remove は重い

イテレータはスナップショット
作ったときの配列を見る。例外にならない

読み込みが多く、書き込みが少ないときに向く。
CopyOnWriteArraySet もある。

Collections.synchronizedList は各メソッドを同期するだけ。
拡張 for の途中で add すると
ConcurrentModificationException。

出典：API CopyOnWriteArrayList・Collections.synchronizedList／JDK 17 での実行結果

ConcurrentHashMap

```
ConcurrentMap<String,Integer> m = new ConcurrentHashMap<>();
// 4 スレッド × 1 万回
m.merge("k", 1, Integer::sum);
m.get("k")              // 40000（原子的）
m.putIfAbsent("k", 0)   // 40000（変えない）
m.putIfAbsent("z", 9)   // null（入れた）

m.put(null, 1);    // NullPointerException
m.put("a", null);  // NullPointerException
new HashMap<>().put(null, null);  // OK
```

メソッド	キーがない	キーがある
putIfAbsent	入れて null	今の値を返す
computeIfAbsent	関数で入れる	何もしない
computeIfPresent	何もしない	関数で更新
compute	v は null	関数で更新
merge	値を入れる	関数で合わせる

> 走査中に put しても例外にならない。

出典：API ConcurrentHashMap・ConcurrentMap／JDK 17 での実行結果

BlockingQueue と SkipList

動き	例外	特別な値	待つ	時間まで待つ
追加	add	offer(false)	put	offer(e, t, 単位)
取り出し	remove	poll(null)	take	poll(t, 単位)

```
BlockingQueue<String> q = new ArrayBlockingQueue<>(1);
q.offer("a") q.offer("b") q.poll() q.poll()
// true false a null
q.add(x) // 満杯なら IllegalStateException
```

```
new ConcurrentSkipListSet<>(List.of("b","c","a"))
// [a, b, c]
new ConcurrentSkipListMap<>(Map.of(3,"c",1,"a"))
// {1=a, 3=c}、firstKey() は 1
```

> SkipList は TreeSet・TreeMap の並行版（自然順序で並ぶ）。

> ArrayBlockingQueue は大きさ固定、LinkedBlockingQueue は指定しなければほぼ無制限。

出典：API BlockingQueue・ArrayBlockingQueue・LinkedBlockingQueue・ConcurrentSkipListSet・ConcurrentSkipListMap／JDK 17 での実行結果

第3章

同期の道具と Flow

待ち合わせと、受ける側に合わせた流れ。

CountDownLatch と CyclicBarrier

CountDownLatch(3)

countDown で 1 ずつ減らす。0 になるまで await で待つ

```
CountDownLatch l = new CountDownLatch(3);
// 3 つのタスクが l.countDown()
l.await(1, TimeUnit.SECONDS)  // true
l.getCount()                   // 0
```

0 になったら戻せない（1 回きり）。

CyclicBarrier(2, action)

2 つのスレッドが await でそろうまで待つ

```
CyclicBarrier b = new CyclicBarrier(2,
  () -> println("all arrived"));
// 2 つのタスクが b.await()
// → all arrived（1 回）→ 両方が進む
```

そろうたびに使い回せる（cyclic）。

出典：API CountDownLatch・CyclicBarrier／JDK 17 での実行結果

Flow のしくみ



インタフェース	役割
Publisher	データを出す
Subscriber	データを受ける
Subscription	request(n)・cancel()
Processor<T,R>	受けて変換して出す（両方）

> 受ける側が量を決める＝背圧（back pressure）。request しなければ onNext は来ない。

出典：API Flow・Flow.Publisher・Subscriber・Subscription・Processor

SubmissionPublisher

```
try (SubmissionPublisher<Integer> p = new SubmissionPublisher<>()) {
  p.subscribe(new Flow.Subscriber<>() {
    Flow.Subscription sub;
    public void onSubscribe(Flow.Subscription s) {
      sub = s; print("sub "); s.request(1); }
    public void onNext(Integer i) {
      got.add(i); sub.request(1); }
    public void onError(Throwable t) {}
    public void onComplete() {
      print("complete "); done.countDown(); }
  });
  p.submit(1); p.submit(2); p.submit(3);
}   // close → 残りが届いてから onComplete
done.await();
// sub complete [1, 2, 3]
```

submit で出す・close で終わり
AutoCloseable なので try-with-resources

配信は別スレッド
既定は ForkJoinPool.commonPool()

> onSubscribe で request(2) だけなら、5 個 submit しても届くのは [1, 2]。

出典：API SubmissionPublisher・Flow.Subscription.request／JDK 17 での実行結果

本番での問われ方

ねらい	見るところ
execute と submit	execute に Callable はコンパイルエラー、submit(Runnable) の get は null
Future の例外	ExecutionException の getCause、TimeoutException、cancel 後の CancellationException
終わらせ方	shutdown 後は RejectedExecutionException、shutdownNow の戻り値、isTerminated
走査中の変更	ArrayList・synchronizedList は CME、CopyOnWriteArrayList はスナップショット
ConcurrentHashMap	null 不可、merge・compute は原子的、putIfAbsent の戻り値
Flow	呼ばれる順、request した数だけ届く

出典：試験内容チェックリスト 5／本資料の問題演習（全 30 問、JDK 17 で確認）